APSC 143: Introduction to Computer Programming for Engineers

Assignment #1 - Functions

Fall 2026

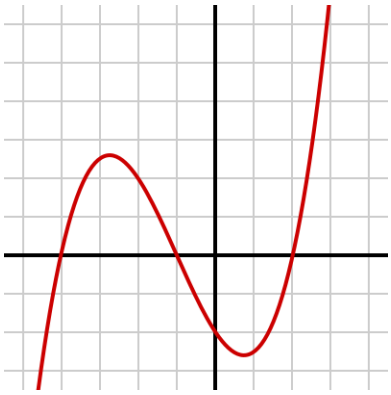


Figure 1: An illustration of a cubic function with three distinct, real-valued roots. Original image source: Wikipedia

1 Introduction

In this assignment, you will have to use and write functions in C. These functions will be combined together to calculate complex mathematical expressions.

Specifically, you will implement functions to solve for the roots of a cubic function,

$$y(x) = x^3 + ax^2 + bx + c, \tag{1}$$

when they are distinct and real valued. The roots of this equation are the values of x that result in $y(x) \equiv 0$. For a cubic equation, there are three roots, x_1, x_2, x_3, although they may not all be real-valued or distinct.

This document contains all the mathematical details necessary to calculate the roots of a cubic equation when it has three distinct, real-valued roots.

To achieve this, you will implement at least five different functions that calculate:

1. The value of a cubic function, given x, a, b, and c, i.e., equation (1) on page 1.
2. The *discriminant* of the cubic equation that tells us how many real-valued roots it has, given a, b, and c, i.e., equation (2) on page 4.
3. The first root of the cubic equation, given a, b, and c, i.e., equation (5) on page 4.

 ©2026 Queen's University

4. The second root of the cubic equation, given a, b, and c, i.e., equation (6) on page 4.
5. The third root of the cubic equation, given a, b, and c, i.e., equation (7) on page 4.

The specific programming requirements for the assignment are provided in Section 2. The specific mathematical details of cubic roots are in Section 3. The instructions on how to submit your work are provided in Section 4.

2 Instructions

In this assignment, you will create a C program and functions to calculate the real-valued roots of cubic equations of the form $y(x) = x^3 + ax^2 + bx + c$. Specifically, you must implement at least the following five functions with exactly these signatures:

1. `double cubicFunction(double a, double b, double c, double x)`
   Calculates and returns the value of the cubic function defined by a, b, and c at x, i.e., equation (1) on page 1.
2. `double discriminant(double a, double b, double c)`
   Calculates and returns the discriminant of the cubic equation defined by a, b, and c, i.e, equation (2) on page 4. When the discriminant is less than zero, the cubic equation has three distinct, real-valued roots.
3. `double cubicRoot1(double a, double b, double c)`
   Calculates and returns the first root of the cubic equation defined by a, b, and c, i.e., equation (5) on page 4.
4. `double cubicRoot2(double a, double b, double c)`
   Calculates and returns the second root of the cubic equation defined by a, b, and c, i.e., equation (6) on page 4.
5. `double cubicRoot3(double a, double b, double c)`
   Calculates and returns the third root of the cubic equation defined by a, b, and c, i.e., equation (7) on page 4.

You will be assessed on the correctness of the output of these five functions. You may choose to implement their internal details however you wish, but **they must have the function signatures (e.g., names, arguments, return values) listed above**.

These required functions share common calculations, and to receive full marks you should use additional functions to appropriately break up the problem and avoid duplicating code.

You will likely find it helpful to also write a `main` function that you can use to test your functions against the example output provided in Table 1. We will not assess your `main` function.

Comments are mandatory for this assignment. Add comments as necessary for key pieces in your code to explain what the program is doing, including any function definitions, assumptions, variable declaration, conditional statements, looping conditions, etc.

 ©2026 Queen's University

The results of your functions must match the sample output (Table 1) when rounded to four decimal places; otherwise, the auto grading software will not be able to grade your assignment, which may affect your mark.

2.1 Assumptions

You can make the following assumptions when writing your functions:

1. `double cubicFunction(double a, double b, double c, double x)`
   The coefficient a is not zero.
2. `double discriminant(double a, double b, double c)`
   The coefficient a is not zero.
3. `double cubicRoot1(double a, double b, double c)`
   The coefficient a is not zero. The cubic equation will have three distinct, real-valued roots, i.e., its discriminant will be less than 0.
4. `double cubicRoot2(double a, double b, double c)`
   The coefficient a is not zero. The cubic equation will have three distinct, real-valued roots, i.e., its discriminant will be less than 0.
5. `double cubicRoot3(double a, double b, double c)`
   The coefficient a is not zero. The cubic equation will have three distinct, real-valued roots, i.e., its discriminant will be less than 0.

2.2 Existing Functions

Like most programming languages, there are a number of existing, useful functions you can use in C. These functions are no different than the ones you will write yourself, other than that they have been rigorously tested and made publicly available to everyone!

Functions provided by `math.h` that you will need for this assignment include:

- `double sqrt(double x)`
  Calculates and returns $\sqrt{x}$ for the given x.
- `double pow(double x, double y)`
  Calculates and returns x^y for the given x and y.
- `double cos(double x)`
  Calculates and returns $\cos(x)$ for the given x in radians.
- `double acos(double y)`
  Calculates and returns $\arccos(y)$ in radians for the given y.
- `M_PI`
  The mathematical constant π, i.e., 3.14159...

You can read more about these functions and `math.h` at https://cplusplus.com/reference/cmath/.

 ©2026 Queen's University

3 Mathematical Background

The roots of a cubic function defined by coefficients a, b, and c are the values of x such that it is equal to zero, i.e., the solutions to the cubic equation,

$$x^3 + ax^2 + bx + c = 0.$$

All cubic equations have three roots, but they may not be distinct and real valued.

A cubic equation is guaranteed to have three distinct, real-valued roots if its discriminant,

$$D = Q^3 + R^2, \tag{2}$$

is strictly less than zero, $D < 0$. The terms Q and R are defined in terms of the cubic function's coefficients as

$$Q = \frac{3b - a^2}{9}, \tag{3}$$

$$R = \frac{9ab - 27c - 2a^3}{54}. \tag{4}$$

If (and only if) the cubic equation has three distinct, real-valued roots (i.e., its discriminant is less than 0), the roots, x_1, x_2, x_3, can be calculated as

$$x_1 = 2\sqrt{(-Q)}\cos\left(\frac{1}{3}\theta\right) - \frac{1}{3}a, \tag{5}$$

$$x_2 = 2\sqrt{(-Q)}\cos\left(\frac{1}{3}\theta + \frac{2}{3}\pi\right) - \frac{1}{3}a, \tag{6}$$

$$x_3 = 2\sqrt{(-Q)}\cos\left(\frac{1}{3}\theta + \frac{4}{3}\pi\right) - \frac{1}{3}a, \tag{7}$$

where

$$\theta = \arccos\left(\frac{R}{\sqrt{(-Q^3)}}\right),$$

- R and Q are defined in equations (3) and (4), and
- π is the familiar mathematical constant (i.e., 3.14159...).

3.1 Guidance and Suggestions

You may work on this assignment however you wish, but below is a guideline on how you may wish to approach this assignment.

In a single `apsc143assign1.c` file:

1. Create signatures for the required functions and an `int main(void)` function.
2. Implement the `cubicFunction`

 ©2026 Queen's University

Table 1: Expected Results for 3 Cubic Functions

a	b	c	discriminant	x_1	x_2	x_3
-4	-9	36	-16.3333	4	-3	3
3	-6	-8	-27.0000	2	-4	-1
-1.5	-1.5	1	-0.4219	2	-1	0.5

3. Call `cubicFunction` in your `main` with specific values of a, b, c, and x, and print the output. Compare the result to what you calculate offline (e.g., using a calculator).
4. Implement the `discriminant` function and any subfunctions it requires.
5. Call `discriminant` in your `main` and print its output to confirm the results with Table 1.
6. Repeat steps 4 & 5 separately for your `cubicRoot1`, `cubicRoot2`, and `cubicRoot3` functions. How many subfunctions can they share between each other and `discriminant`?
7. Test your `cubicFunction` and `cubicRoot` functions by confirming in your `main` that the `cubicFunction` evaluates to 0.0 at the value found by a `cubicRoot` function.

Some additional hints that may help you complete the assignment:

- If when you compile your program you get the error:

  ```
  error:  'M_PI' undeclared
  ```

  then first make sure you have included `math.h` at the top of your `apsc143assign1.c` file. If you have and the error persists, then add the following to your `apsc143assign1.c` right before your first line of code:

  ```
  #ifndef M_PI
  #define M_PI 3.14159265358979323846264338
  #endif
  ```

- If when you compile your program you get the errors:

  ```
  undefined reference to 'sqrt'
  undefined reference to 'pow'
  undefined reference to 'cos'
  undefined reference to 'acos'
  ```

  then first make sure you have included `math.h` at the top of your `apsc143assign1.c` file and then add the following line to the end of your `CMakeLists.txt`:

  ```
  target_link_libraries(EXECUTABLE_NAME m)
  ```

  where `EXECUTABLE_NAME` should be replaced by the name of your executable, which you will also see in the preceding line of the form: `add_executable(EXECUTABLE_NAME apsc143assign1.c)`.

 ©2026 Queen's University

4 Submission Instructions

Create your program and upload it to Gradescope for grading. Your program file **must be named** `apsc143assign1.c` in order for your assignment to be graded. Do not include any personal information (student number, name, etc.) in your submission.

Include a comment in your code attesting to the originality of your work and that you have not used Generative AI.

Refer to the assignment rubric below for a detailed breakdown of the grading criteria. Your submission must adhere to the assignment rules as outlined in the submission policy document for this course, which can also be found on OnQ.

There is zero tolerance for plagiarism in this course. The auto grading software will automatically flag potential cases of plagiarism, which will be reviewed by the teaching team.

More information on assignment submissions can be found on OnQ, and information on the specific definition and repercussions of plagiarism can be found in the "Begin Here (About This Course)" module.

4.1 Assignment Rubric

Test case marks are based purely on code output when given specific inputs. Code design, style, and commenting are graded by the teaching team looking at your code. Use comments effectively to not only get full marks in the comments section, but to also make your code clearer to the marking TA.

A portion of each assignment's marks will be given for your understanding of your code. If your code only uses the concepts taught so far in the course, then you will receive these marks automatically. If your code uses other concepts, including those not yet presented or not covered in the course, then in order to receive these marks you may be asked to explain to a member of the teaching team how your code works and why you chose to solve the problem that way.

 ©2026 Queen's University

Table 2: Marking Rubric — Output

Test Cases (Marks will only be awarded if the code compiles)	**Level 3 (3 points)**	**Level 2 (2 points)**	**Level 1 (1 points)**	**N/A (0 points)**	**Criterion Score**
Assignment Correctness - `cubicFunction`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `discriminant`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `cubicRoot1`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `cubicRoot2`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `cubicRoot3`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - Hidden Test Cases	Actual output perfectly matches expected output.	Some of the required values are correct.	Few values are correct.	The code did not compile or crashed during runtime.	/3

©2026 Queen's University

Table 3: Marking Rubric — Design

Code Design (Marks will only be awarded if the code compiles)	**Level 3 (6 points)**	**Level 2 (4 points)**	**Level 1 (2 points)**	**Level 0 (0 points)**	**Criterion Score**
Function Organization	Code is organized effectively into subfunctions to balance code reuse and complexity.	Code is organized into subfunctions but minor code reuse remains or functions result in minor unnecessary additional code complexity.	Code is incompletely organized into subfunctions with major code reuse or functions are vastly over applied and code is unnecessarily complex.	Code is not organized into subfunctions and significant code reuse exists.	/6

©2026 Queen's University

Table 4: Marking Rubric — Style

Code Style	**Level 2 (2 points)**	**Level 1 (1 points)**	**Level 0 (0 points)**	**Criterion Score**
Coding Style	The coding style is consistent, clear, and well organized.	1 or more instances exist where coding style is not considered.	The coding style lacks clarity.	/2
Commenting	Code is well documented. Comments provide clarity for all tasks and logic.	Few comments are used, major tasks are undocumented and logic is difficult to follow. Comments included are vague and ineffective.	No meaningful comments are used.	/2

©2026 Queen's University

Table 5: Marking Rubric — Understanding

Code Understanding (Marks will only be awarded if the code compiles)	**Level 3 (9 points)**	**Level 2 (6 points)**	**Level 1 (3 points)**	**Level 0 (0 points)**	**Criterion Score**
Code Understanding	Student uses concepts covered in class that are assumed to be understood or uses concepts not covered in the class and can clearly explain and justify their submitted code to a member of the teaching team.	Student uses concepts not covered in class and can explain their submitted code to a member of the teaching team. Their justification of aspects of their submitted code to a member of the teaching team lacks clarity.	Student uses concepts not covered in class. Their explanation and justification of aspects of their submitted code to a member of the teaching team lacks clarity.	Student uses concepts not covered in class and cannot explain nor justify aspects of their submitted code to a member of the teaching team.	/9

©2026 Queen's University